Jeudi 29/10 — Versionner son travail avec Git

Kevin Bardoux

jeudi 29 octobre 2026

Table des matières

Objectifs 2
Matin — Git : versionner son travail (démonstrations Visualizing Git) 2
Rappel (5 min) 2
Apport 1 — Versionner : les trois zones de Git (10 min) 2
Trois zones — attention et vérification 4
Démonstration — Visualizing Git (10 min) 4
Apport 2 — Les commandes de base (15 min) 4
Commandes — attention et vérification 6
Exercice guidé — Mon premier dépôt (50 min) 6
Apport 2 (suite) — Un bon message de commit (5 min) 9
Message — attention et vérification 10
À retenir — ce matin 10
Débrief 11
Après-midi — TP — « Git Journal » · Learn Git Branching 11
Apport 3 — Ignorer des fichiers (5 min) 11
`.gitignore` — attention et vérification 12
TP « Git Journal » (1 h 30) 12
Learn Git Branching (1 h 30) 12
Pas à pas — le niveau 1 (15 min) 13
À retenir — cet après-midi 21
Si le temps le permet — Le graphe de votre journal (15 min) 21
Pour aller plus loin 23

Sources 23

> Note
>
> **Compétences visées** : C1 · résolution de problème **Livrable** : le dépôt `git-journal` avec au moins 8 commits aux messages clairs.

Objectifs

- Expliquer à quoi sert un gestionnaire de versions.
- Enregistrer son travail par petits commits, avec des messages clairs.
- Lire l'historique et les différences entre deux versions.
- Annuler une modification qui n'est pas encore enregistrée.

Comment se passe la journée. Le matin, deux notions : les trois zones de Git, puis les commandes de base, avec le message du commit. L'après-midi, le fichier `.gitignore`, juste avant le Git Journal qui s'en sert. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine chaque demi-journée.

Matin — Git : versionner son travail (démonstrations Visualizing Git)

Rappel (5 min)

1. Comment afficher aussi les fichiers cachés ?
2. Quelle clé SSH peut-on partager ?
3. Que veut dire « portée » d'une variable ?

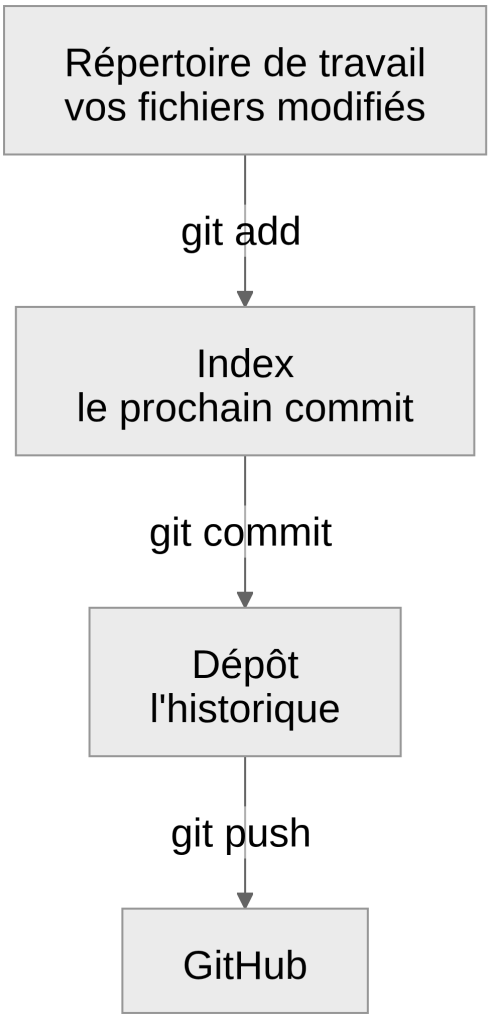
Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — Versionner : les trois zones de Git (10 min)

Pourquoi. Garder l'historique de chaque étape d'un projet, revenir en arrière après une erreur, travailler à plusieurs sans s'écraser :

c’est le rôle d’un gestionnaire de versions. Le référentiel attend l’usage d’un outil de type Git (Ministère du Travail 2023) ; Git fait partie des outils obligatoires de la formation.

Définition. Un gestionnaire de versions enregistre l’évolution d’un ensemble de fichiers au cours du temps, pour pouvoir rappeler une version antérieure (Scott Chacon et Ben Straub, s. d.-a). Git range vos fichiers dans trois zones : le **répertoire de travail** (vos fichiers, tels que vous les modifiez), l’**index** (ce que vous préparez pour le prochain commit, avec `git add`) et le **dépôt** (l’historique des commits enregistrés, avec `git commit`) (Scott Chacon et Ben Straub, s. d.-c). Un **commit** est une photo enregistrée de vos fichiers, avec un message.



Un fichier passe du répertoire de travail à l’index (git add), puis au dépôt (git commit) ; git push l’envoie sur GitHub.

Pas à pas. C'est la démonstration Visualizing Git, ci-dessous : cet outil dessine le graphe des commits à mesure que l'on tape des commandes Git (Git School, s. d.).

Trois zones — attention et vérification

> ⚠ Attention
>
> Un fichier modifié mais pas ajouté (`git add`) n'entre pas dans le commit. Avant chaque commit, `git status` montre ce qui sera enregistré.

- **Vérifiez** : quelle commande fait passer un fichier dans l'index ?

Démonstration — Visualizing Git (10 min)

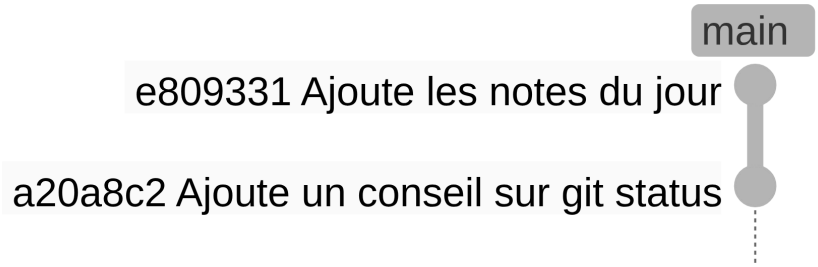
1. Le formateur tape des commandes : le schéma se dessine.
2. Chaque `git commit` ajoute un rond à l'historique.
3. À vous : trois commits, et observez le schéma.

Apport 2 — Les commandes de base (15 min)

- **Pourquoi** : enregistrer, relire et annuler, en quelques commandes.
- **`git status`** : la commande à taper le plus souvent.

Pourquoi. Chaque jour, vous enregistrez votre travail, vous regardez ce qui a changé, et parfois vous annulez une erreur : sept commandes suffisent.

Définition. `git init` crée un dépôt dans le dossier ; `git status` dit où vous en êtes ; `git add` prépare un fichier ; `git commit -m "…"` enregistre ; `git log --oneline` montre l'historique ; `git diff` montre ce qui a changé depuis le dernier commit ; `git restore` annule une modification pas encore préparée. Voir les chapitres « Enregistrer des modifications dans le dépôt » (Scott Chacon et Ben Straub, s. d.-c), « Visualiser l'historique des validations » (Scott Chacon et Ben Straub, s. d.-e) et « Annuler des actions » (Scott Chacon et Ben Straub, s. d.-b) du livre Pro Git, ainsi que la documentation de `git restore` (Git, s. d.).



L'historique de l'exercice guidé : chaque commit est un rond, avec son identifiant et son message ; main désigne le dernier. git log –oneline affiche la même liste, du plus récent au plus ancien.

Exemple, commenté ligne par ligne.

```
git init
git status
git add notes.md
git commit -m "Ajoute les notes du jour"
git log --oneline
git diff
git restore notes.md
```

Ligne 1

`git init` crée un dépôt dans le dossier courant : un dossier caché, `.git`, qui gardera tout l'historique. Une seule fois par projet.

Ligne 2

`git status` dit où vous en êtes : la branche, les fichiers modifiés, ceux qui sont dans l'index. Tapez-la avant chaque commit.

Ligne 3

`git add notes.md` place le fichier dans l'index : il fera partie du prochain commit.

Ligne 4

`git commit -m "…"` enregistre le contenu de l'index dans le dépôt, avec le message écrit entre guillemets.

Ligne 5

`git log --oneline` affiche l'historique, un commit par ligne : son identifiant court, puis son message.

Ligne 6

`git diff` montre ce qui a changé depuis le dernier commit et n'est pas encore dans l'index : `+` devant une ligne ajoutée, `-` devant une ligne retirée.

Ligne 7

`git restore notes.md` remet le fichier dans l'état du dernier commit : la modification est effacée, sans retour.

Pas à pas. C'est l'exercice guidé, ci-dessous : le formateur fait chaque étape au vidéoprojecteur, vous la refaites aussitôt. Un message de Git vous bloque ? La fiche Lire un message d'erreur explique les plus fréquents.

Commandes — attention et vérification

> ⚠ Attention
>
> `git restore` efface vos modifications non enregistrées, sans retour possible. Relisez `git diff` avant de l'utiliser.

- **Vérifiez** : quelle commande montre ce qui a changé ?

Exercice guidé — Mon premier dépôt (50 min)

Niveau : guidé.

1. Créez `dwwm/essai-git`, entrez-y (`cd`), puis `git init`.
2. Dans VS Code, créez `notes.md` ; puis `git add` et `git commit`.
3. Modifiez `notes.md` ; `git diff` ; puis un deuxième commit.
4. Modifiez encore, puis annulez avec `git restore notes.md`.
5. **Réussi si** : `git log --oneline` montre 2 commits.

À l'étape 2, écrivez une seule ligne dans `notes.md` : `# Notes du jour`. À l'étape 3, ajoutez une ligne vide, puis `- Taper git status avant chaque commit.`

Vous devez voir, étape par étape. Les identifiants des commits (`e809331`, `a20a8c2`) seront différents chez vous ; une ligne vide sépare chaque commande de la suivante.

Étape 1 :

```
stagiaire@PC MINGW64 ~/dwwm/essai-git
$ git init
Dépôt Git vide initialisé dans C:/Users/stagiaire/dwwm/
essai-git/.git/
```

L'invite affiche maintenant `(main)` : vous êtes dans un dépôt, sur la branche `main`.

Étape 2, avant puis après `git add` :

```
stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git status
Sur la branche main

Aucun commit

Fichiers non suivis:
  (utilisez "git add <fichier>..." pour inclure dans ce
qui sera validé)
    notes.md

aucune modification ajoutée à la validation mais des
fichiers non suivis sont présents (utilisez "git add"
pour les suivre)

stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git add notes.md

stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git status
Sur la branche main

Aucun commit

Modifications qui seront validées :
  (utilisez "git rm --cached <fichier>..." pour
désindexer)
    nouveau fichier : notes.md
```

```
stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git commit -m "Ajoute les notes du jour"
[main (commit racine) e809331] Ajoute les notes du jour
 1 file changed, 1 insertion(+)
 create mode 100644 notes.md
```

Étape 3 : `git diff` marque d'un `+` les deux lignes ajoutées (la ligne vide et le conseil).

```
stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git diff
diff --git a/notes.md b/notes.md
index 6e74760..27ba1f6 100644
--- a/notes.md
+++ b/notes.md
@@ -1 +1,3 @@
 # Notes du jour
+
+- Taper git status avant chaque commit.

stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git add notes.md

stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git commit -m "Ajoute un conseil sur git status"
[main a20a8c2] Ajoute un conseil sur git status
 1 file changed, 2 insertions(+)
```

Étape 4 : après une modification, `git status` propose lui-même `git restore` ; après `git restore`, plus rien à valider.

```
stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git status
Sur la branche main
Modifications qui ne seront pas validées :
  (utilisez "git add <fichier>..." pour mettre à jour
ce qui sera validé)
```

```
  (utilisez "git restore <fichier>..." pour annuler les
modifications dans le répertoire de travail)
    modifié :         notes.md

aucune modification n'a été ajoutée à la validation
(utilisez "git add" ou "git commit -a")

stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git restore notes.md

stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git status
Sur la branche main
rien à valider, la copie de travail est propre
```

Étape 5 : les deux commits, le plus récent en haut.

```
stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git log --oneline
a20a8c2 Ajoute un conseil sur git status
e809331 Ajoute les notes du jour
```

Un avertissement sous Windows. Si un fichier a été créé dans Git Bash (par exemple avec `echo`), `git add` peut afficher, en anglais :

```
warning: in the working copy of 'notes.md', LF will be
replaced by CRLF the next time Git touches it
```

Ce n'est pas une erreur : le fichier est bien ajouté, continuez. Windows ne termine pas les lignes avec les mêmes caractères invisibles que macOS et Linux (CRLF d'un côté, LF de l'autre) ; Git for Windows les convertit tout seul (GitHub, s. d.). Un fichier créé dans VS Code, sous Windows, n'affiche pas cet avertissement.

Apport 2 (suite) — Un bon message de commit (5 min)

- **Pourquoi** : relire l'historique dans six mois, et le jury aussi.

- **Message** : ce que fait le commit, à l'impératif, en une ligne.

Pourquoi. L'historique raconte votre projet. Des messages clairs permettent de retrouver une étape, de comprendre une erreur, et de montrer votre méthode au jury.

Définition. Un bon message dit **ce que** fait le commit, à l'impératif (« Ajoute… », « Corrige… »), en moins de 70 caractères. Un commit correspond à une étape cohérente, pas à une journée entière.

Exemple.

```
Bon     : Ajoute l'entrée de la semaine 2 au journal
Bon     : Corrige la faute de frappe du titre
À éviter : modif
À éviter : ça marche
```

Message — attention et vérification

> ⚠ Attention
>
> `git commit` sans `-m` ouvre un éditeur dans le terminal (souvent Vim). Pour en sortir : `Échap` (windows), `Échap` (mac), puis `:wq`, puis Entrée.

- **Vérifiez** : réécrivez « modif » pour un commit qui ajoute un lien.

À retenir — ce matin

> À retenir
>
> - **Trois zones** : `git add` prépare, `git commit` enregistre.
> - **`git status`** avant tout ; **`git restore`** efface sans retour.
> - **Message** : à l'impératif, en une ligne ; un commit par étape.

Le détail, notion par notion :

- **Trois zones** : répertoire de travail, index, dépôt ; `git add`, puis `git commit` ; `git push` (demain) envoie sur GitHub.
- **Commandes** : `git init`, `git status`, `git add`, `git commit -m`, `git log --oneline`, `git diff`, `git restore`.

- **Message de commit** : ce que fait le commit, à l'impératif, en une ligne ; un commit par étape.

Débrief

- Quelle différence entre `git add` et `git commit` ?
- Que montre `git diff` ?
- Quand utiliser `git restore`, et pourquoi être prudent ?

Après-midi — TP — « Git Journal » · Learn Git Branching

Apport 3 — Ignorer des fichiers (5 min)

- **Pourquoi** : ne pas enregistrer ce qui se réinstalle ou ce qui est secret.
- **`.gitignore`** : la liste de ce que Git doit ignorer.

Pourquoi. Le dossier `node_modules` pèse lourd et se réinstalle avec `npm install` ; un fichier de mots de passe ne doit jamais partir sur GitHub.

Définition. Le fichier `.gitignore`, à la racine du dépôt, liste les fichiers et dossiers que Git ne suit pas : ils n'apparaissent plus dans `git status` (Scott Chacon et Ben Straub, s. d.-c).

Exemple.

```
node_modules/
.env
```

Pas à pas.

1. Dans VS Code, créez le fichier `.gitignore` à la racine de `essai-git`.
2. Écrivez `node_modules/` sur la première ligne. Enregistrez.
3. `git status` : `.gitignore` apparaît, à ajouter et à enregistrer comme les autres.

Vous devez voir, à l'étape 3 :

```
stagiaire@PC MINGW64 ~/dwwm/essai-git (main)
$ git status
```

```
Sur la branche main
Fichiers non suivis:
  (utilisez "git add <fichier>..." pour inclure dans ce
qui sera validé)
    .gitignore

aucune modification ajoutée à la validation mais des
fichiers non suivis sont présents (utilisez "git add"
pour les suivre)
```

`.gitignore` — attention et vérification

> ⚠ Attention
>
> `.gitignore` n'efface pas un fichier déjà enregistré dans un commit : écrivez-le **avant** le premier commit du fichier à ignorer.

- **Vérifiez** : `git status` montre-t-il `node_modules` ?

TP « Git Journal » (1 h 30)

Niveau : semi-guidé.

1. Créez le dépôt `dwwm/git-journal` avec un fichier `journal.md`.
2. Pour chaque semaine (1 à 4) : ce que vous avez appris, en 3 lignes.
3. Avant chaque commit : `git status`, puis `git diff`.
4. Un commit par entrée ; ajoutez un `.gitignore`.
5. **Réussi si** : au moins **8 commits** aux messages clairs.

Idées de commits : un par semaine, un pour le `.gitignore`, un pour corriger une faute, un pour ajouter un lien vers une documentation. Provoquez aussi une erreur volontaire (une ligne effacée par mégarde) et réparez-la avec `git restore` avant de la valider. Vendredi, ce dépôt sera publié sur GitHub.

Learn Git Branching (1 h 30)

Niveau : guidé.

1. Ouvrez Learn Git Branching, en français.
2. Faites la séquence d'**introduction** : ses quatre niveaux.

3. Commits, branches, fusion : notez une phrase par notion.
4. Puis la séquence suivante, aussi loin que possible.

Learn Git Branching est un jeu qui fait taper de vraies commandes Git et dessine le résultat (Peter Cottle, s. d.). La séquence d'introduction compte quatre niveaux : les commits, les branches, la fusion (*merge*) et le rebase. Le rebase est donné pour information : nous ne l'utiliserons pas dans la formation. Les branches et la fusion servent dès demain avec GitHub (Scott Chacon et Ben Straub, s. d.-d).

Pas à pas — le niveau 1 (15 min)
Chaque étape donne une action, puis ce que vous devez voir. Sur chaque capture, l'endroit à cliquer est entouré d'un cadre jaune, avec le numéro de l'étape. Quand la capture montre tout l'écran, un zoom sur la zone utile la suit. Sur téléphone, touchez une capture pour l'agrandir.

Étape 1. Ouvrez Learn Git Branching, version du cours dans votre navigateur. Cette copie est hébergée avec le cours, sans mesure d'audience ni publicité ; votre progression reste dans votre navigateur.

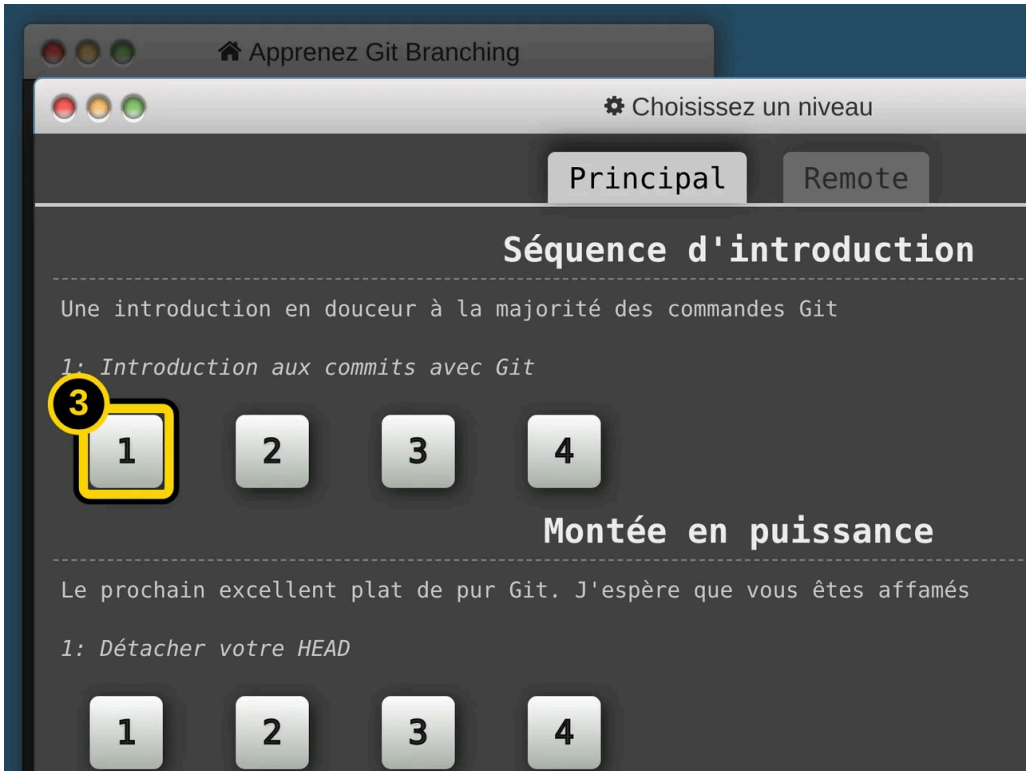
Vous devez voir : une fenêtre grise intitulée « Bienvenue sur Learn Git Branching ! ».

Étape 2. Cliquez sur l'icône rouge de sortie, en bas de la fenêtre, à gauche de la flèche verte.



Vous devez voir : une fenêtre intitulée « Choisissez un niveau », avec la rubrique « Séquence d'introduction » en haut.

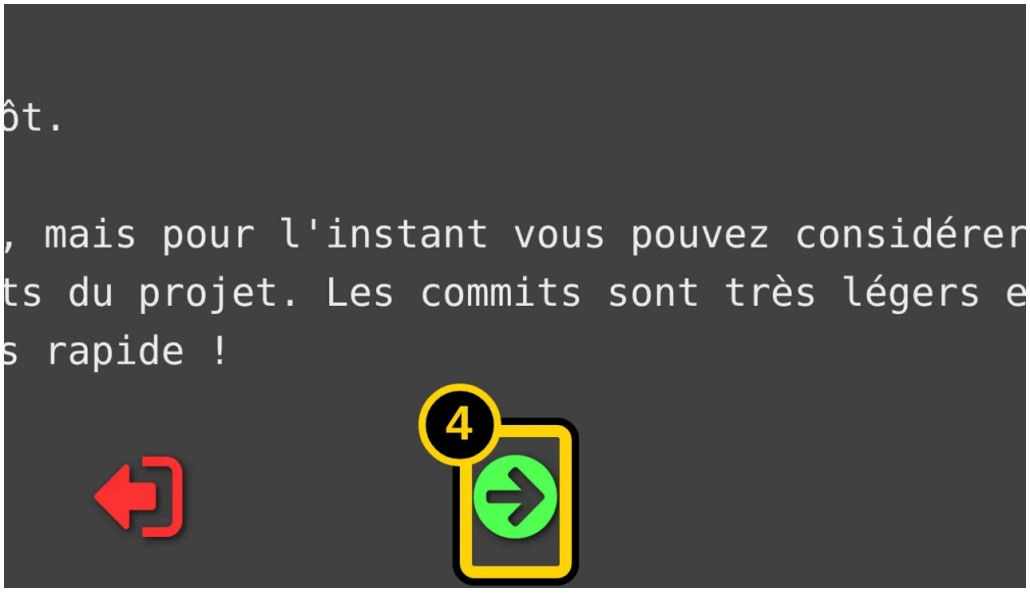
Étape 3. Sous « Séquence d'introduction », cliquez sur le carré **1**.



Vous devez voir : une fenêtre intitulée « Commits Git », qui explique ce qu'est un commit.

Étape 4. Lisez le texte, puis cliquez sur la flèche verte, en bas de la fenêtre.





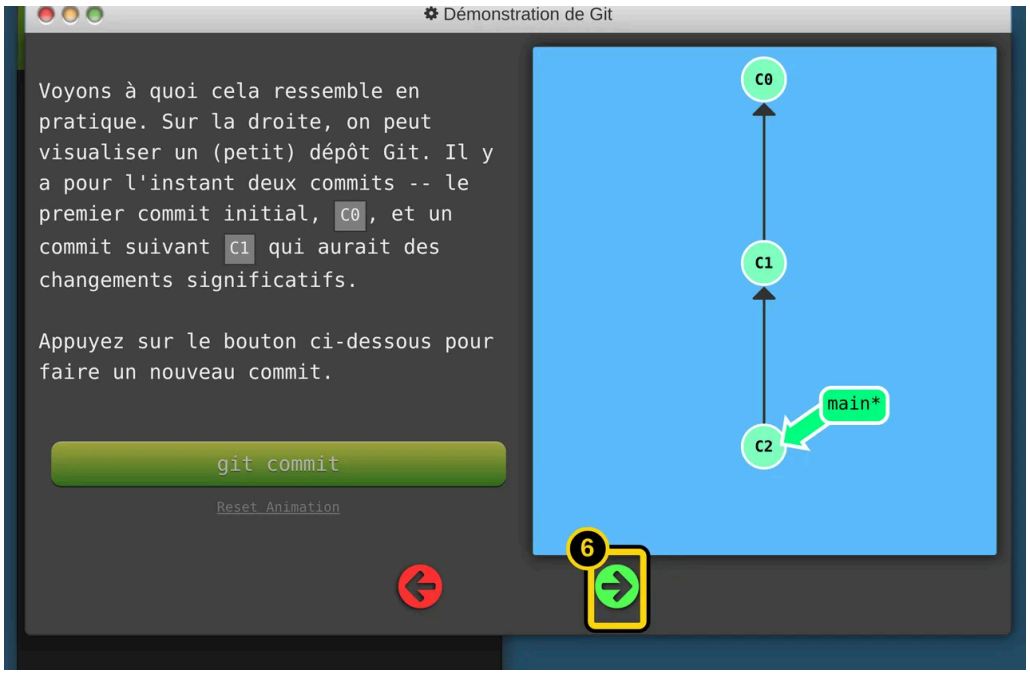
Vous devez voir : une fenêtre intitulée « Démonstration de Git ». À droite, un dessin bleu montre deux ronds, C0 et C1, reliés par une flèche ; l'étiquette `main*` pointe sur C1.

Étape 5. Cliquez sur le grand bouton vert `git commit`, à gauche.



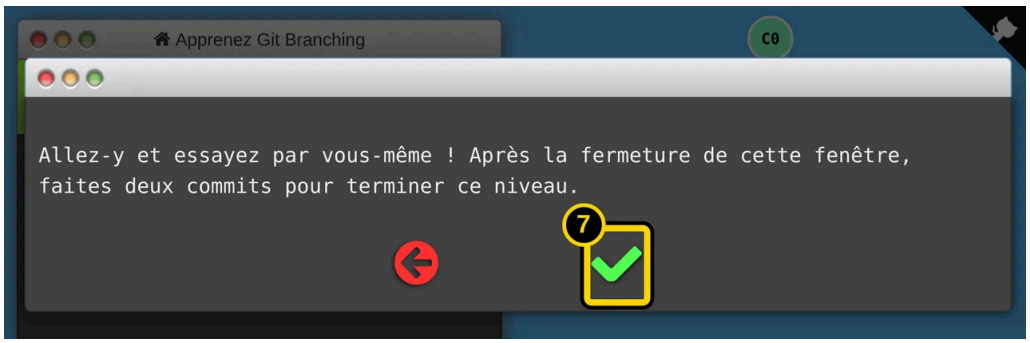
Vous devez voir : un nouveau rond, C2, apparaît sous C1 ; l'étiquette `main*` pointe maintenant sur C2.

Étape 6. Cliquez sur la flèche verte, en bas de la fenêtre.



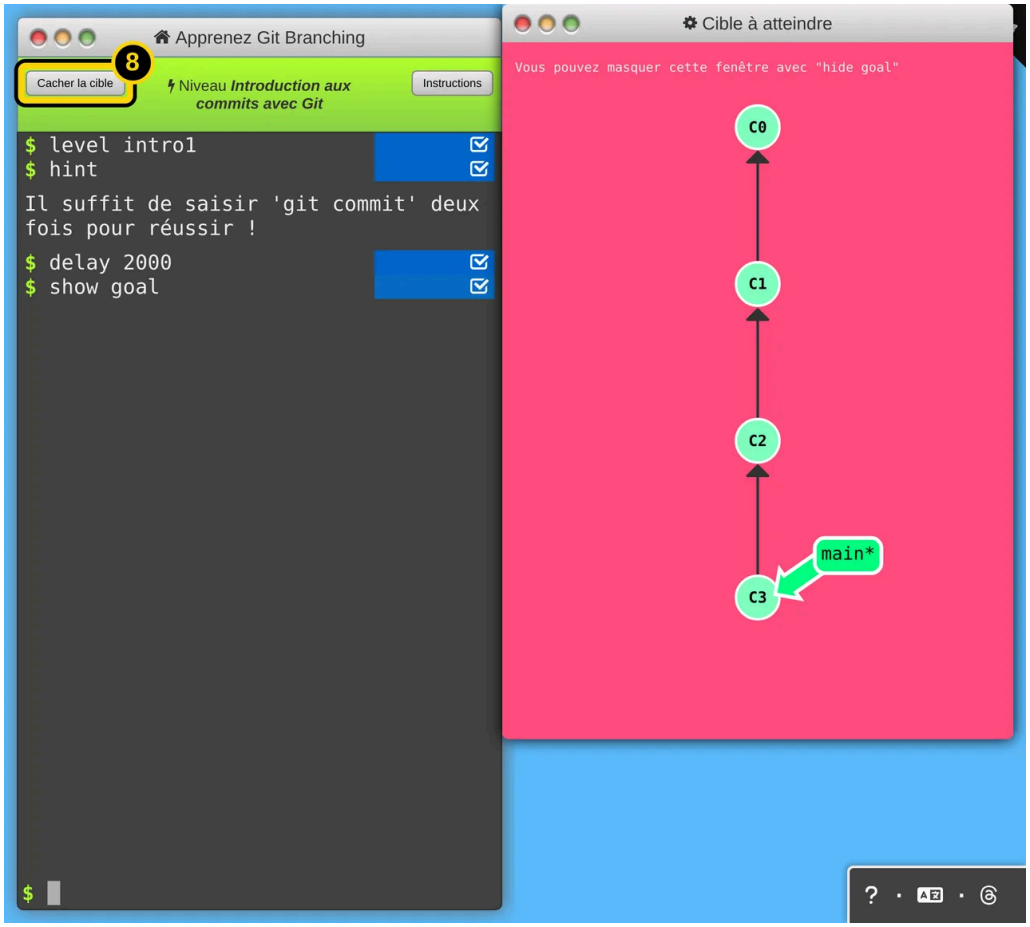
Vous devez voir : une petite fenêtre qui dit « Allez-y et essayez par vous-même ! », avec une coche verte.

Étape 7. Cliquez sur la coche verte.



Vous devez voir : à gauche, un terminal noir avec une barre verte en haut ; à droite, une fenêtre rose « Cible à atteindre » qui montre quatre ronds, de C0 à C3.

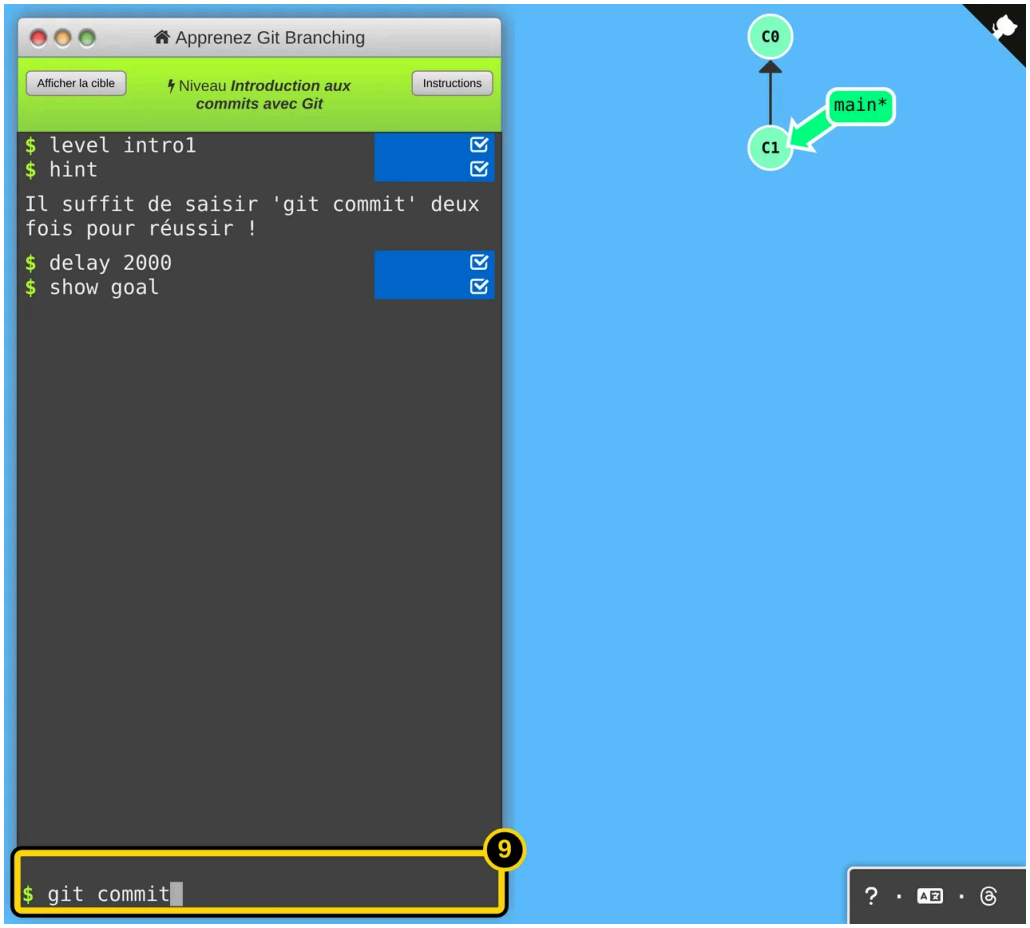
Étape 8. Dans la barre verte du terminal, cliquez sur le bouton « Cacher la cible ».

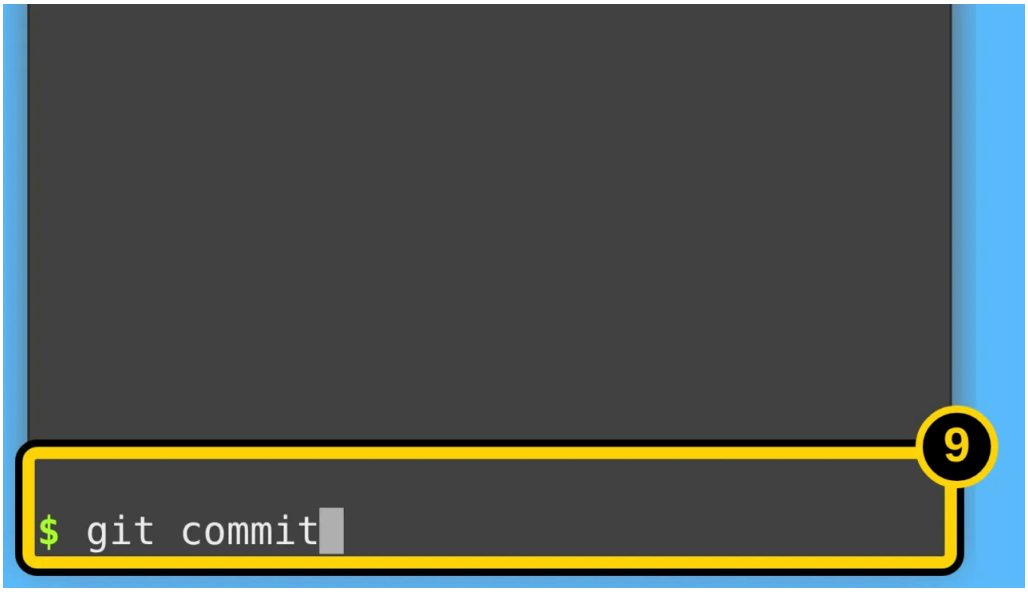




Vous devez voir : la fenêtre rose disparaît ; sur le fond bleu, deux ronds, C0 et C1, et l'étiquette `main*` sur C1.

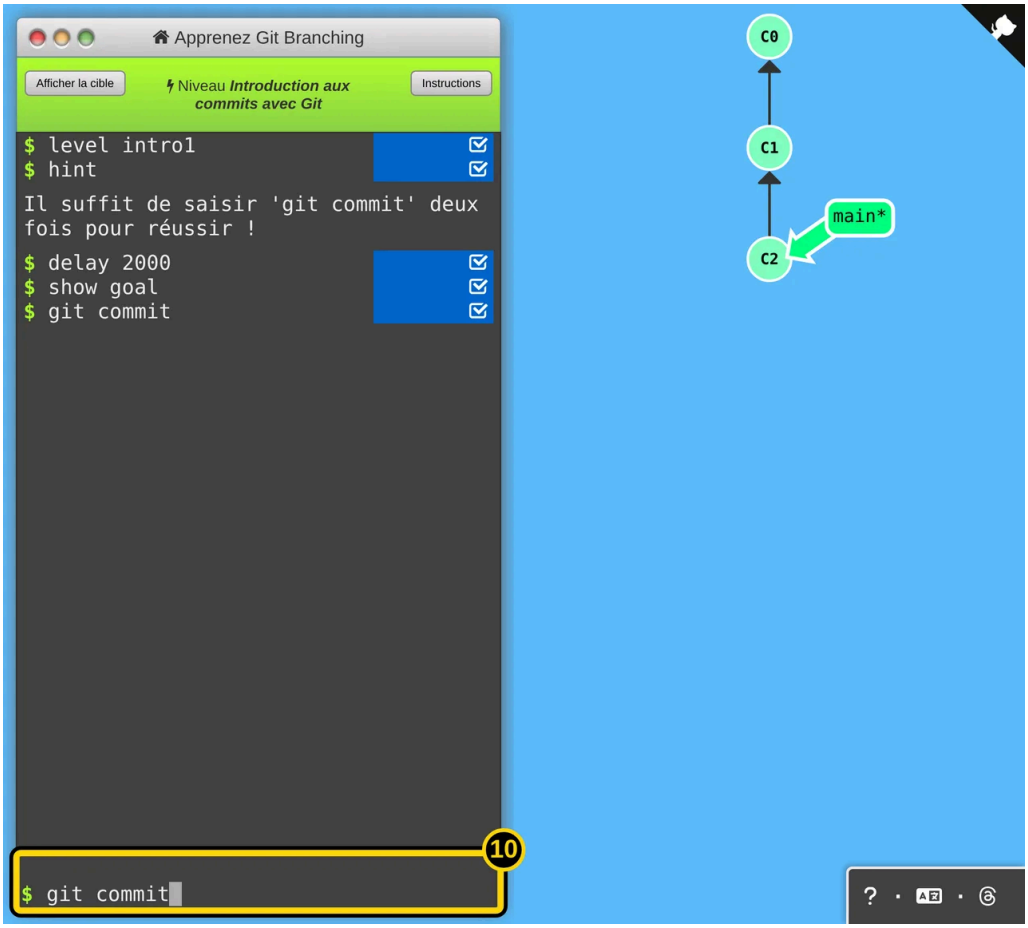
Étape 9. Tapez `git commit` (le texte s'écrit en bas du terminal, après le `$`), puis appuyez sur `Entrée`.

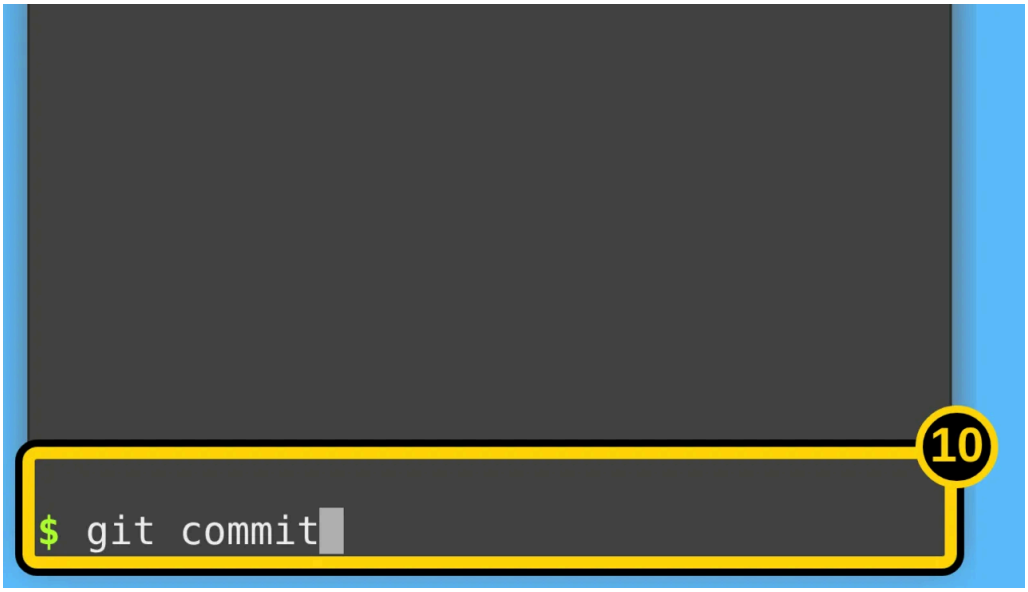




Vous devez voir : la ligne `git commit` remonte dans le terminal, avec une case cochée ; sur le dessin, un rond C2 apparaît sous C1.

Étape 10. Tapez de nouveau `git commit`, puis appuyez sur `Entrée`.





Vous devez voir : un rond C3 apparaît, le texte « Résolu !!! » s'affiche, puis une fenêtre « Beau Travail!!! » propose le niveau suivant, « Gérer les branches avec Git ».

Étape 11. Cliquez sur le bouton vert « Confirmer », en bas à droite.



Vous devez voir : une fenêtre intitulée « Branches Git » : c'est le niveau 2.

Les niveaux 2, 3 et 4 se jouent de la même façon : lisez les fenêtres (flèche verte), essayez les démonstrations (bouton vert), fermez avec la coche verte, puis tapez les commandes demandées. Si vous êtes bloqués, tapez `reset` pour recommencer le niveau, ou `levels` pour revenir à la liste des niveaux.

Captures : Learn Git Branching, Peter Cottle, licence MIT ; la version du cours est construite à partir du même code.

À retenir — cet après-midi

> À retenir
>
> - `.gitignore` : ce que Git ignore, écrit avant le premier commit.
> - **Avant chaque commit** : `git status`, puis `git diff`.
> - **Branche** (découverte) : une ligne de travail à part, demain sur GitHub.

Le détail : `.gitignore` liste ce que Git ne suit pas (`node_modules/`, `.env`) ; il n'efface pas un fichier déjà enregistré dans un commit.

Critères de réussite

- ☐ `git log --oneline` montre au moins 8 commits dans `git-journal`
- ☐ Chaque message décrit clairement son étape
- ☐ Un `.gitignore` est présent
- ☐ Les quatre niveaux d'introduction de Learn Git Branching sont faits

Si le temps le permet — Le graphe de votre journal (15 min)
Niveau : guidé. Vous travaillez dans le dépôt `git-journal` créé cet après-midi au TP « Git Journal » (dossier `dwwm/git-journal`). Il doit contenir **au moins 2 commits**.

1. Dans Git Bash, placez-vous dans le dépôt : `cd ~/dwwm/git-journal`. **Vous devez voir** : l'invite se termine par `(main)`.
2. Tapez `git log --oneline`. **Vous devez voir** : au moins 2 lignes, une par commit. Moins de 2 : terminez d'abord le TP « Git Journal ».
3. Tapez `git log --oneline --graph --all`. **Vous devez voir** : une étoile `*` devant chaque commit, toutes alignées : une seule ligne, sans branche.
4. Créez une branche d'essai et passez dessus : `git switch -c essai`. **Vous devez voir** : `Basculement sur la nouvelle`

`branche 'essai'`, et l'invite se termine par `(essai)`. Les branches sont expliquées demain ; Learn Git Branching vous en a montré le dessin.

5. Dans VS Code, ajoutez une ligne à la fin de `journal.md` ; enregistrez. Puis : `git add journal.md` et `git commit -m "Essaie une branche"`.
6. Revenez sur la branche principale : `git switch main`. **Vous devez voir** : l'invite se termine par `(main)` ; la ligne ajoutée à l'étape 5 a disparu de `journal.md` : elle est sur la branche `essai`.
7. Dans VS Code, créez le fichier `idees.md` avec une ligne ; enregistrez. Puis : `git add idees.md` et `git commit -m "Ajoute une liste d'idées"`.
8. Tapez à nouveau `git log --oneline --graph --all`. **Vous devez voir** : deux chemins qui se séparent après le même commit, l'un marqué `(essai)`, l'autre `(HEAD -> main)`, comme ci-dessous.
9. Comparez avec les dessins de Learn Git Branching : chaque `*` est un rond, chaque `|` et `/` un trait.

```
stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git log --oneline --graph --all
* a53e060 (essai) Essaie une branche
| * 1aa7eb6 (HEAD -> main) Ajoute une liste d'idées
|/
* 583899e Ajoute la semaine 1
* ceadcd5 Ajoute le journal
```

Vos identifiants (`a53e060`…) et vos messages sont différents ; la forme compte. La branche `essai` peut rester : elle ne gêne pas l'exercice de demain.

> ⚠ Attention
>
> `git log` n'affiche rien, ou affiche `fatal: votre branche actuelle 'main' ne contient encore aucun commit` : vous n'êtes pas dans `git-journal`, ou aucun commit n'y est enregistré. Tapez `pwd`, puis `git status`, et suivez l'entrée « git log ne montre rien » de la fiche « Lire un message d'erreur ».

Pour aller plus loin

- Demandez à CS50.ai un quiz sur les trois zones.

Sources

Git. s. d. « git-restore Documentation ». Consulté le 26 septembre 2026. https://git-scm.com/docs/git-restore/fr.

Git School. s. d. « Visualizing Git ». Consulté le 26 septembre 2026. https://git-school.github.io/visualizing-git/.

GitHub. s. d. « Configuring Git to handle line endings ». Consulté le 17 octobre 2026. https://docs.github.com/en/get-started/git-basics/configuring-git-to-handle-line-endings.

Ministère du Travail. 2023. *Référentiel emploi activités compétences du titre professionnel Développeur web et web mobile (REAC TP-01280)*.

Peter Cottle. s. d. « Learn Git Branching ». Consulté le 26 septembre 2026. https://learngitbranching.js.org/?locale=fr_FR.

Scott Chacon et Ben Straub. s. d.-a. « Pro Git : À propos de la gestion de version ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/D%C3%A9marrage-rapide-%C3%80-propos-de-la-gestion-de-version.

Scott Chacon et Ben Straub. s. d.-b. « Pro Git : Annuler des actions ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-bases-de-Git-Annuler-des-actions.

Scott Chacon et Ben Straub. s. d.-c. « Pro Git : Enregistrer des modifications dans le dépôt ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-bases-de-Git-Enregistrer-des-modifications-dans-le-d%C3%A9p%C3%B4t.

Scott Chacon et Ben Straub. s. d.-d. « Pro Git : Les branches en bref ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-branches-avec-Git-Les-branches-en-bref.

Scott Chacon et Ben Straub. s. d.-e. « Pro Git : Visualiser l’historique des validations ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-bases-de-Git-Visualiser-l%E2%80%99historique-des-validations.